

Goats vs. Trolls: Object-Oriented Programming with Java

1. Introduction & Learning Objectives

This lecture explores object-oriented design principles using a simplified RPG-style system. The focus is on inheritance, polymorphism, abstraction, and UML modeling rather than game mechanics.

2. Domain Analysis

We begin with domain analysis to identify shared state and behavior among game entities. All goats share hit points, healing logic, and consciousness state, while differing in attack and damage handling.

3. Enumerations: DamageType

The DamageType enum represents a fixed set of possible damage categories such as physical, magical, poison, holy, and elemental. Enums improve safety and readability by preventing invalid values.

4. Modeling Attacks

Attacks are modeled as objects rather than primitive values. Each Attack includes a name, an array of hit values, and a damage type. This design supports complex attacks such as multi-hit spells.

5. Abstract Parent Class: Goat

The Goat class encapsulates all shared state and behavior. It is declared abstract to prevent instantiation of meaningless base goats and to enforce implementation of combat behavior in subclasses.

6. Mage Class

Mage extends Goat and represents a magic-focused character. Mages have lower health, deal magical damage, resist magic, and are vulnerable to physical attacks.

7. Fighter Class

Fighter extends Goat and represents a physical combatant. Fighters have higher health, deal physical damage, resist physical attacks, and are vulnerable to magical damage.

8. Adding New Goat Types

New goat types such as Cleric can be added by extending Goat and implementing attack and takeDamage. No changes to the battle system are required, demonstrating polymorphism.

9. Polymorphism and GoatArena

The GoatArena class contains a single battle method that accepts Goat parameters. At runtime, Java dynamically dispatches the correct subclass implementations.

10. UML Class Diagram

A UML class diagram visually represents the inheritance hierarchy, abstract methods, and relationships between Goat, its subclasses, Attack, and DamageType.

11. Key Takeaways

- Inheritance eliminates duplication
- Polymorphism enables flexible design
- Abstract classes enforce correctness
- Enums model fixed categories
- Composition improves expressiveness